CS246: Database Management Systems Lab

SQL programming

Vijaya saradhi

Department of Computer Science and Engineering
Indian Institute of Technology Guwahati

MySQL Stored Programs

- Stored programs is a generic term used for stored procedure, stored functions and triggers
- Without stored programs database system cannot claim full compliance with variety of standards including ANSI/ISO
- These standards describe how a DBMS should execute stored programs.
- Judicial use of stored programs lead to greater database security and integriety
- Improve overall application performance
- Improve maintainability

What is Stored Program?

What is it anyway?

- A computer program
- A series of instructions associated with a name
- The source code and any compiled version of the stored program are held within database server's system tables
- Program is executed within the memory address of database server

What is Stored Program?

Stored Procedures

Invocation	A generic program unit that is executed on request
Parameters	Accepts multiple input and output parameters

What is Stored Program?

Stored Procedures

Invocation	A generic program unit that is executed on request
Parameters	Accepts multiple input and output parameters

Stored Functions

Similar	to stored procedures
Constraint	Execution results in the return of single value
Invocation	Can be used within standard SQL statements
Extend SQL	Use of functions in SQL statements amount to extending SQL functionality

What is Stored Program?

Stored Procedures

Invocation	A generic program unit that is executed on request
Parameters	Accepts multiple input and output parameters

Stored Functions

Similar	to stored procedures
Constraint	Execution results in the return of single value
Invocation	Can be used within standard SQL statements
Extend SQL	Use of functions in SQL statements amount to extending SQL functionality

Triggers

Invocation	Activated in response to an activity within the database
DML	In particular when INSERT, UPDATE or DELETE statements are used

Why use Stored Programs?

Why another language?

- Developers have multitude of programming languages from which to choose
- Many of these are not database languages
- The code written in these languages does not reside in or managed by database server
- Stored programs offer many advantages. These are

Why use Stored Programs?

Advantages of Stored Programs

- Can lead to more secure database
- Offer mechanism to abstract data access routines in turn improve the maintainability of code as data structures evolve
- Reduces network traffic; Work on the data from within the server rather than transferring data across network
- Can be used to implement Common routines accessible from multiple applications
- They can be executed either within the database server
- Database-centric logic can be isolated in stored programs

Types of variables

- User-defined variables
- Parameter and local variables
- System variables
- Global variables

User defined variables - 01

- Denoted with @var_name. Example 1 set @my_first_variable = 0;
- Example 2: set @v1 = X'41';
- Example 3: set @v2 = X'41' + 0;
- Example 4: set @v3 = CAST(X'41' AS UNSIGNED);

```
mysql> SET @t1=1, @t2=2, @t3:=4;
mysql> SELECT @t1, @t2, @t3, @t4 := @t1+@t2+@t3;
```

```
+------+------+------+--------------------+
| @t1  | @t2  | @t3  | @t4 := @t1+@t2+@t3 |
+------+------+------+--------------------+
|    1 |    2 |    4 |                  7 |
+------+------+------+--------------------+
```

User defined variables - 02

Don'ts with user defined variables

Never assign a value to a user variable and read the value within the same statement. Here the order of evaluation for expressions involving user variables is undefined.

SELECT @a, @a:=@a+1, ...;

User defined variables - 03

Don'ts with user defined variables

The default result type of a variable is based on its type at the start of the statement.

```
mysql> SET @a='test';
mysql> SELECT @a,(@a:=20) FROM tbl_name;
```

System defined variables - 01

- System-defined variables in MySQL are predefined variables that store system-related information and configurations.
- Variables that configure MySQL operations.
- A system variable can have a global value that affects server operation as a whole OR
- A session value that affects the current session OR
- both the are possible.
- System variables are dynamic and can be changed at runtime using the SET statement to affect operation of the current server instance
- If you change a session system variable, the value remains in effect within your session until you change the variable to a different value or the session ends.
- The change has no effect on other sessions.

System defined variables - 02

- Example: SELECT @@version;
- Example: SELECT @@autocommit;
- Example: SELECT @@max_connections;

Global variable - 01

- A global variable in MySQL is a system-defined variable that applies to the entire MySQL server and affects all sessions
- Values assigned to global variables is visible to any client that accesses it.
- If you change a global system variable, the value is remembered and used to initialize the session value for new sessions until you change the variable to a different value or the server exits.
- Examples: `wait_timeout`, `sql_select_limit`, etc.

Global variable - 02

```
SHOW GLOBAL VARIABLES;
SET GLOBAL max_connections = 1000;
SET @@GLOBAL.max_connections = 1000;
```

Default value

To set a global system variable value to the compiled-in MySQL default value set the variable to the value DEFAULT.

```
SET @@SESSION.max_join_size = DEFAULT;
SET @@SESSION.max_join_size = @@GLOBAL.max_join_size;
```

System variable - 02

system variables

To assign a value to a session system variable, precede the variable name by the SESSION OR LOCAL keyword, by the @@SESSION., @@LOCAL., or @@ qualifier, or by no keyword or no modifier at all

```
SET SESSION sql_mode = 'TRADITIONAL';
SET LOCAL sql_mode = 'TRADITIONAL';
SET @@SESSION.sql_mode = 'TRADITIONAL';
SET @@LOCAL.sql_mode = 'TRADITIONAL';
SET @@sql_mode = 'TRADITIONAL';
SET sql_mode = 'TRADITIONAL';
```

Multiple assignments

A SET statement can contain multiple variable assignments, separated by commas

```
SET @x = 1, SESSION sql_mode = '';
```

Multiple variables in one statement

```
SET GLOBAL sort_buffer_size = 1000000,
                    SESSION sort_buffer_size = 1000000;
SET @@GLOBAL.sort_buffer_size = 1000000,
                    @@LOCAL.sort_buffer_size = 1000000;
SET GLOBAL max_connections = 1000,
                     sort_buffer_size = 1000000;
```

Variables in stored programs - 01

- System variables, global variables and user-defined variables can be used in stored programs just as they can be used outside stored-program context.
- In addition, stored programs can use DECLARE to *define local variables*, and stored routines (procedures and functions) can be declared to take parameters that communicate values between the routine and its caller

Variables in stored programs - 02

- To declare local variables, use the DECLARE statement
- Variables can be set directly with the SET statement
- Results from queries can be retrieved into local variables using `SELECT ... INTO var_list`

Local variable DECLARE statement

Example

Declaration DECLARE `variable_name` datatype;

Example `DECLARE first_var INT;`

Value `first_var` is initialized with $\perp$ (NULL)

Example `DECLARE first_var INT DEFAULT 0;`

Value `first_var` is initialized with value 0

Local variable DECLARE statement

Example

Declaration DECLARE `variable_name` datatype;

Example `DECLARE first_var INT;`

Value `first_var` is initialized with $\perp$ (NULL)

Example `DECLARE first_var INT DEFAULT 0;`

Value `first_var` is initialized with value 0

More examples

- `DECLARE var1 INT DEFAULT -20000;`
- `DECLARE var2 FLOAT DEFAULT 1.8e-8;`
- `DECLARE var3 DOUBLE DEFAULT 2e45;`
- `DECLARE var4 DATE DEFAULT '1999-12-31';`

Assigning Values to local/system/global variables

Example - 1

```
SET variable_name = expression;
SET var1 = 10;
```

Example - 2

```
SET variable_name = expression;
SET var2 = 10.0001;
```

Example - 3

```
SET variable_name = expression;
SET var4 = '2018-11-12';
```

Parameters - 1

Procedures and Functions

Are variables that can be passed into or out of the stored program

IN parameter

- An IN parameter passes a value into a procedure.
- The procedure might modify the value, but the modification is not visible to the caller when the procedure returns.

Parameters - 2

OUT parameter

- An OUT parameter passes a value from the procedure back to the caller.
- Its initial value is NULL within the procedure, and its value is visible to the caller when the procedure returns.

INOUT parameter

An INOUT parameter is initialized by the caller, can be modified by the procedure, and any change made by the procedure is visible to the caller when the procedure returns.

Example - 01

```
mysql> delimiter //

mysql> CREATE PROCEDURE citycount (IN country CHAR(3), OUT cities INT)
       BEGIN
         SELECT COUNT(*) INTO cities FROM world.city
         WHERE CountryCode = country;
       END//
Query OK, 0 rows affected (0.01 sec)

mysql> delimiter ;

mysql> CALL citycount('JPN', @cities); -- cities in Japan
Query OK, 1 row affected (0.00 sec)

mysql> SELECT @cities;
+----------+
| @cities  |
+----------+
|      248 |
+----------+
1 row in set (0.00 sec)
```

Parameter - IN

Example

```
DELIMITER //
CREATE PROCEDURE demoIN(IN var1 INT)
BEGIN
        -- See the value of IN parameter
        SELECT var1;

        -- Modify
        SET var1 = 2;

        -- See the value of IN parameter
        SELECT var1;
END;//
DELIMITER ;
```

Execution

```
mysql> SET @myvar = 1;
mysql> CALL demoIN(@myvar);
mysql> SELECT @myvar;
```

- First line initializes @myvar variable
- Second line calls the stored procedure `demoIN`
- Within `demoIN` var1 is read containing value 1
- Within `demoIN` var1 is modified to value 2
- Third line read the variable @myvar which is 1

Parameter - OUT

Example

```
DELIMITER //
CREATE PROCEDURE demoOUT(OUT var1 INT)
BEGIN
        -- See the value of OUT parameter
        SELECT var1;

        -- Modify
        SET var1 = 2;

        -- See the value of OUT parameter
        SELECT var1;
END;//
DELIMITER ;
```

Execution

```
mysql> SET @myvar = 1;
mysql> CALL demoOUT(@myvar);
mysql> SELECT @myvar;
```

- First line initializes @myvar variable
- Second line calls the stored procedure `demoOUT`
- Withing `demoOUT` var1 is read containing value NULL (irrespective of its initialization outside procedure)
- Withing `demoOUT` var1 is modified to value 2
- Third line read the variable @myvar which is 2

Parameter - INOUT

Example

```
DELIMITER //
CREATE PROCEDURE demoINOUT(INOUT var1 INT)
BEGIN
        -- See the value of INOUT parameter
        SELECT var1;

        -- Modify
        SET var1 = 2;

        -- See the value of INOUT parameter
        SELECT var1;
END;//
DELIMITER ;
```

Execution

```
mysql> SET @myvar = 1;
mysql> CALL demoINOUT(@myvar);
mysql> SELECT @myvar;
```

- First line initializes @myvar variable
- Second line calls the stored procedure `demoINOUT`
- Withing `demoINOUT` var1 is read containing value 1
- Withing `demoINOUT` var1 is modified to value 2
- Third line read the variable @myvar which is 2

Built-in Functions

Categories

String functions Perform string manipulation; concatenation of two strings, obtaining substring etc

Mathematical functions Example: trigonometric functions, random number functions, logarithms etc

Date and time functions add or subtract time intervals from dates; find difference between two dates etc

Miscellaneous functions every thing not easily categorized in the above three groupings; encryption functions etc

String functions

```
SELECT roll_number, CONCAT(sur_name, " ", first_name, " ", last_name) as full_name
FROM   Student
WHERE  Dept = 'EEE';
```

Mathematical functions

```
SELECT roll_number, ABS(quiz1_marks)
FROM   Student
WHERE  Dept = 'BSBE';
```

Mathematical functions

```
SELECT roll_number, ROUND(SPI, 2)
FROM   Student
WHERE  Dept = 'EEE';
```

Date and time functions

```
SELECT roll_number, DAYNAME(held_on)
FROM   Attendance
WHERE  cid = 'CS441M';
```

Date and time functions

```
SELECT DATE_ADD('2018-05-01',INTERVAL 1 DAY);
-- '2018-05-02'

SELECT DATE_SUB('2018-05-01',INTERVAL 1 YEAR);
-- '2017-05-01'

SELECT DATE_ADD('2020-12-31 23:59:59', INTERVAL 1 SECOND);
-- '2021-01-01 00:00:00'

SELECT DATE_ADD('2018-12-31 23:59:59', INTERVAL 1 DAY);
-- '2019-01-01 23:59:59'
```

Block structure of stored programs

- Stored program consists of one or more blocks
- Each block commences with a BEGIN statement and terminate by an END
- Blocks are useful for defining variables within a block
- Variable within a block are not visible outside the block

Block structure

- Various types of declarations can appear in a block
- Order in which these can occur is as follows
- Variable and condition declarations (errors)
- Cursor declarations
- Handler declarations
- Program code
- Violation of this order results in error

Block structure - order

```
[label:] BEGIN
        variable declarations
        condition declarations
        cursor declarations
        handler declarations

        program code
END [label];
```

Block structure - Example

```
DELIMITER //
CREATE PROCEDURE f1()
BEGIN
        DECLARE var1 INT DEFAULT 10;
END;//
DELIMITER ;
```

Nested Blocks

Nested block structures

- Some instances needed nested block structures
- Blocks that are defined within an enclosing block
- Variables defined within a block are not available outside the block
- However the variables are visible to blocks that are declared within the block

Nested block structure - Example

```
DELIMITER //
CREATE PROCEDURE f1()
BEGIN
        DECLARE outer_variable INT DEFAULT 10;
        BEGIN
                DECLARE  inner_variable INT DEFAULT 20;
                SET inner_variable = 22;
        END;

        SET outer_variable = 12;

END;//
DELIMITER ;
```

Nested block structure - Example

```
DELIMITER //
CREATE PROCEDURE f2()
BEGIN
        DECLARE outer_variable INT DEFAULT 10;
        BEGIN
                DECLARE  inner_variable INT DEFAULT 20;
                SET inner_variable = 22;
        END;
        SET outer_variable = 12;
        SELECT inner_variable, 'This statement causes an error';

END; //
DELIMITER ;
```

Nested Blocks - Overriding variables

Nested block structure - Example

```
DELIMITER //
CREATE PROCEDURE f3()
BEGIN
        DECLARE outer_variable INT DEFAULT 10;
        SET outer_variable = 27;
        BEGIN
                SET outer_variable = 57;
        END;
        SELECT outer_variable, 'This statement causes overwriting on 27 with 57';
END;//
DELIMITER ;
```

Nested block structure - Example

Changes made to an overloaded variable in an inner block are not visible outside the block

```
DELIMITER //
CREATE PROCEDURE f4()
BEGIN
        DECLARE my_variable varchar(20);
        SET my_variable='This value was set in the OUTER block';

        BEGIN
                DECLARE  my_variable varchar(20);
                SET my_variable='This value was set in the INNER block';
        END;

        SELECT my_variable, 'Can''t see changes made in the INNER block';
        SELECT 'As the scope of INNER BLOCK is ended';
END;//
DELIMITER ;
```

LEAVE statement

Exiting nested blocks

```
DELIMITER //
CREATE PROCEDURE f5()
outer_block: BEGIN

        DECLARE l_status INT;
        SET l_status=1;

        inner_block: BEGIN
                IF (l_status = 1)
                THEN
                        LEAVE inner_block;
                END IF
                SELECT 'This_statement_will_never_be_executed';
        END inner_block;
        SELECT 'End_of_program';
END outer_block;//
DELIMITER ;
```

Conditional Statement - IF

```
DELIMITER //
CREATE PROCEDURE s_AND_d(IN sale_id INT, IN sale_value FLOAT)
BEGIN
        IF( sale_value > 200 )
        THEN
                CALL apply_free_shipping(sale_id);

                IF ( sale_vale > 500 )
                THEN
                        CALL apply_discount(sale_id, 20);
                END IF;

        END IF;

END; //
DELIMITER ;


mysql> SELECT customer_name, s_AND_d(sale_id, sale_value) FROM  Customer;
```

Conditional Statement - IF

```
DELIMITER //
CREATE PROCEDURE f6(IN cpi FLOAT)
BEGIN
        IF( cpi > 7.0 )
        THEN

                SELECT roll_number, full_name
                FROM   Student
                WHERE  Dept = 'EEE';

        ELSE IF ( cpi BETWEEN 5.0 AND 7.0 )
        THEN

                SELECT roll_number, full_name
                FROM   Student
                WHERE  Dept = 'BSBE';

        ELSE

                SELECT roll_number, full_name
                FROM   Student
                WHERE  Dept <> 'BSBE' AND Dept <> 'EEE';

        END IF;
END; //
DELIMITER ;
```

Conditional Statement - CASE

Functionally equivalent to IF - ELSE IF - ELSE - END block

```
CASE
        WHEN condition THEN
                statements
         [WHEN condition THEN
                statements]
         [ELSE
                statements]
END CASE;
```

Conditional Statement - CASE

```
DELIMITER //
CREATE PROCEDURE f7(IN sale_value FLOAT, IN customer_status
                    ENUM(PLATINUM, GOLD, SILVER, BRONZE), IN sale_id INT)
BEGIN
        DECLARE dummy INT DEFAULT -1;
        CASE
                WHEN (sale_value > 200 AND customer_status = PLATINUM ) THEN
                        CALL free_shipping(sale_id);
                        CALL apply_discount(sale_id, 20);
                WHEN (sale_value > 200 AND customer_status = GOLD ) THEN
                        CALL free_shipping(sale_id);
                        CALL apply_discount(sale_id, 15);
                WHEN (sale_value > 200 AND customer_status = SILVER ) THEN
                        CALL free_shipping(sale_id);
                        CALL apply_discount(sale_id, 10);
                WHEN (sale_value > 200 AND customer_status = BRONZE ) THEN
                        CALL free_shipping(sale_id);
                        CALL apply_discount(sale_id, 5);
                WHEN (sale_value > 200 ) THEN
                        CALL free_shipping(sale_id);
                ELSE
                        SET dummy = 0;
        END CASE; //
DELIMITER ;
```

Iterative Processing with Loops

- LOOP statement
- REPEAT ... UNTIL
- WHILE

LOOP

Syntax

```
[label:] LOOP
        statements
END LOOP [label];
```

Example

```
DELIMITER //
CREATE PROCEDURE f7()
BEGIN
        DECLARE i INT DEFAULT 1;
        SET i = 1;
        myloop: LOOP
                SET i = i + 1;
                IF i = 10
                THEN
                        LEAVE myloop;
                END IF;
        END LOOP myloop;
        SELECT 'I can count 10';
END; //
DELIMITER ;
```

REPEAT ... UNTIL

Syntax

```
[label:] REPEAT
        statements
UNTIL expression
END REPEAT [label]
```

REPEAT ... UNTIL

Example

```
DELIMITER //
CREATE PROCEDURE f8()
BEGIN
        DECLARE i INT DEFAULT 1;
        SET i = 0;
        loop1: REPEAT

                SET i = i + 1;

                IF MOD(i, 2) <> 0
                THEN
                        SELECT CONCAT(i, " is an ODD number");
                END IF;

        UNTIL i >= 10;

        END REPEAT loop1;
END; //
DELIMITER;
```

WHILE Loop

Syntax

```
[label:] WHILE expression
DO
        statements
END WHILE [label]
```

WHILE Statement

Example

```
DELIMITER //
CREATE PROCEDURE f9()
BEGIN
        DECLARE i INT DEFAULT 1;
        SET i = 1;
        loop1: WHILE i <= 10 DO

                IF MOD(i, 2) <> 0
                THEN
                        SELECT CONCAT(i, " is ODD number");
                END IF;
                SET i = i + 1;

                END WHILE loop1;
END; //
DELIMITER;
```

Nested loops

Example

```
DELIMITER //
CREATE PROCEDURE f10()
BEGIN
        DECLARE i INT DEFAULT 1;
        DECLARE j INT DEFAULT 1;
        outer_loop: LOOP
                SET j = 1;
                inner_loop: LOOP
                        SELECT CONCAT(i, " times ", j, " is ", i * j);
                        SET j = j + 1;
                        IF j > 12
                        THEN
                                LEAVE inner_loop;
                        END IF;
                END LOOP inner_loop;

                SET i = i + 1;
                IF i > 12
                THEN
                        LEAVE outer_loop;
                END IF;
        END outer_loop;
END; //
DELIMITER ;
```

Example

```
DELIMITER //
CREATE PROCEDURE simple_sqls()
BEGIN
        DECLARE i INT DEFAULT 1;

        DROP TABLE IF EXISTS test_table;
        CREATE TABLE test_table(id INT, some_data CHAR(30), PRIMARY KEY (id));

        WHILE ( i <= 10 )
        DO
                INSERT INTO test_table(i, CONCAT("record", i));
                SET i = i + 1;
        END WHILE;

END; //
DELIMITER ;
```

Cursor - I

Impedance Model Mis-match

- SQL always returns relations
- Other programming languages has data types that are not relations
- These languages cannot hold relations returned by SQL
- C language has pointers; where as SQL do not have any such construct
- As a result, passing data between SQL and other languages is not straightforward
- Mechanisms must be devised to allow the development of programs that use both SQL and other languages

Cursor - I

Impedance Model Mis-match

- Versatile way to connect SQL queries to a host language is with a cursor
- Cursor runs through the tuples of a relation
- This relation can be stored table, or it can be something that is generated by a query

Cursor - I

Details

- SELECT will return a relation
- Returned relation will not be stored
- Often the need to process one row at a time of returned relation arise
- Cursor helps examining one row at a time

Cursor - II

Details

- Assume the returned relation to be a file in itself
- Operations required for reading a file are
  - Declare file pointer
  - Open the file
  - Read one line at a time repeatedly
  - close the file
- Similar tasks are associated with cursor

Cursor - III

Declare cursor

```
DECLARE cursor_name CURSOR FOR SELECT   statement;

OPEN    cursor_name;

FETCH cursor_name INTO variable_list;

CLOSE cursor_name
```

Cursor - Example

Example

```
DELIMITER //
CREATE PROCEDURE f11()
BEGIN
-- Declare variables
        DECLARE i INT DEFAULT 1;
        DECLARE sno INT;
        DECLARE sname char(50);
        DECLARE rating INT DEFAULT 10;
        DECLARE age INT DEFAULT 16;

-- Declare cursors
        DECLARE my_first_cursor CURSOR FOR
        SELECT   *
        FROM     Sailors
        WHERE    age > 20 AND rating BETWEEN 5 AND 7;

-- Declare cursor handler
        DECLARE CONTINUE HANDLER FOR NOT FOUND SET NO_records = 1;
```

Cursor - Example

Example

```
        OPEN my_first_cursor;

-- loop through all the rows
loop_1: REPEAT
-- Get one roll number from list of registered students into variable rn
                FETCH my_first_cursor INTO (sno, sname, rating, age);
-- Check number of records in the cursor
                IF NO_records = 1 THEN
                        LEAVE loop_1;
                END IF;

                UNTIL NO_records
        END REPEAT loop_1;
        CLOSE my_first_cursor;
END; //
DELIMITER ;
```

Scrolling

- Cursor gives us flexibility as how to move through the tuples of the relation
- The default choice is to start at the beginning of the relation and fetch the tuples in order
- Fetch all tuples until end of the relation
- Other orders in which tuples may be fetched
- These options are not available in `MySQL` yet we will discuss these

Scrolling

- Instruct the cursor to open in `SCROLL` model before the keyword `CURSOR`
- `EXEC SQL DECLARE name SCROLL CURSOR FOR MovieExec;`
- This will tell SQL that cursor may be used in a manner other than moving in forward direction alone
- The FETCH is responsible for specifying the direction from which the next tuple be obtained
  - `FETCH NEXT` retrieve next tuple
  - `FETCH PRIOR` retrieve previous tuple
  - `FETCH FIRST` retrieve first tuple
  - `FETCH LAST` retrieve last tuple
  - `FETCH ABSOLUTE i` specifies the position of the tuple to be fetched from the top of the relation

Complex Integrity Check

Registered Students

row_id	cid	roll_number	name	email
1	BT101	1234	ABC	abc
2	BT101	1235	ABD	abd
3	BT101	1236	ABE	abe
...	...	...	...	...
1	BT102	1234	ABC	abc
2	BT102	1235	ABD	abd
3	BT102	1236	ABE	abe

Exam timings

cid	held_on	st	et
BT101	25-Nov-2018	09:00	12:00
BT102	26-Nov-2018	09:00	12:00
...	...	...	...

Complex Integrity Check - Part I

Example

```
-- Skip regular delimiter
delimiter //

CREATE PROCEDURE tt_violation()
BEGIN

  -- Declare variables
  DECLARE result_set INT default 0;
  DECLARE rn VARCHAR(10);
  DECLARE no_records INT default 0;

  -- Declare cursor
  -- Get distinct roll numbers, irrespective of credited course
  DECLARE student_courses_cursor CURSOR FOR \
    SELECT      DISTINCT(roll_number)
    FROM        student_list
    GROUP BY    roll_number;

  -- Declare cursor handler
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET NO_records = 1;
```

Complex Integrity Check - Part II

Example

```
-- open cursor for reading one row at a time
OPEN student_courses_cursor;

-- loop through all the rows
loop_1: REPEAT

-- Get one roll number from list of registered students into variable rn
FETCH student_courses_cursor INTO rn;

-- Check number of records in the cursor
IF NO_records = 1 THEN
  LEAVE loop_1;
END IF;

-- All course id's and associated exam times for this student are placed in this table
CREATE TABLE rn1234 AS (
    SELECT  row_id, roll_number, student_list.cid, held_on, st, et
    FROM    student_list
    JOIN    timetable
    ON      student_list.cid = timetable.cid
    WHERE   roll_number=rn
  );
```

Example

Single student's course registration table

row_id	roll_number	cid	held_on	st	et
1	1234	BT101	25-Nov-2018	09:00	12:00
2	1234	BT102	26-Nov-2018	09:00	12:00
3	1234	BT103	27-Nov-2018	09:00	12:00
4	1234	BT202	25-Nov-2018	09:00	12:00

Complex Integrity Check - Part III

Example

```
-- Count: two distinct course have same exam times
SELECT COUNT(*) INTO result_set
FROM   rn1234 AS r1
JOIN   rn1234 AS r2
WHERE  r1.row_id <> r2.row_id
AND    r1.cid <> r2.cid
AND    r1.held_on = r2.held_on
AND    r1.st = r2.st
AND    r1.et = r2.et;
-- List the time table clashing courses for this student
IF result_set > 0
THEN
  SELECT 'exam time table vilation for roll number ', rn;
  SELECT *
  FROM   rn1234 AS r1
  JOIN   rn1234 AS r2
  WHERE r1.row_id <> r2.row_id
  AND    r1.cid <> r2.cid
  AND    r1.held_on = r2.held_on
  AND    r1.st = r2.st
  AND    r1.et = r2.et;
END IF;
```

Complex Integrity Check - Part IV

Example

```
        DROP TABLE rn1234;

      UNTIL NO_records
    END REPEAT loop_1;
    CLOSE student_courses_cursor;
END; //

DELIMITER ;
```

SQL exception - 01

- An SQL system indicates error conditions by setting non-zero sequence of digits in SQLSTATE
- Example 02000 `no tuple found`
- Example 21000 `single row select has returned more than one row`
- We can declare user defined exceptions called exception handler
- Invoked whenever one of a list of these error codes appear in SQLSTATE during execution of a statement
- Each exception handler is associated with a block of code
- delineated by `BEGIN` ... `END`

Exceptions - 02

- When an error occurs inside a stored procedure, it is important to handle it appropriately, such as continuing or exiting the current code block's execution, and issuing a meaningful error message.
- MySQL provides an easy way to define handlers that handle from general conditions such as warnings or exceptions to specific conditions e.g., specific error codes.

```
DECLARE action HANDLER FOR condition_value statement;
```

- If a condition whose value matches the condition_value, MySQL will execute the statement and continue or exit the current code block based on the action.
- The action accepts one of the following values

  CONTINUE the execution of the enclosing code block `BEGIN ... END`

  EXIT the execution of the enclosing code block, where the handler is declared, terminates.
- The condition_value specifies a particular condition or a class of conditions that activate the handler. The condition_value accepts one of the following values
  - A MySQL error code

```
DECLARE action HANDLER FOR condition_value statement;
```

- The condition_value specifies a particular condition or a class of conditions that activate the handler. The condition_value accepts one of the following values
  - A `MySQL` error code
  - A standard `SQLSTATE` value. Or it can be an `SQLWARNING` , `NOTFOUND` or `SQLEXCEPTION` condition, which is shorthand for the class of `SQLSTATE` values. The `NOTFOUND` condition is used for a cursor or `SELECT INTO variable_list` statement.
  - A named condition associated with either a `MySQL` error code or `SQLSTATE` value.

Example

The following handler set the value of the hasError variable to 1 and continue the execution if an SQLEXCEPTION occurs

```
DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
SET hasError = 1;
```

Example

The following handler sets the value of the RowNotFound variable to 1 and continues execution if there is no more row to fetch in case of a cursor or SELECT INTO statement:

```
DECLARE CONTINUE HANDLER FOR NOT FOUND
SET RowNotFound = 1;
```

Example

If a duplicate key error occurs, the following handler issues an error message and continues execution.

```
DECLARE CONTINUE HANDLER FOR 1062
SELECT 'Error, duplicate key occurred';
```

SupplierProducts

supplierId	producerId

```
DELIMITER |
CREATE PROCEDURE InsertSupplierProduct(
    IN inSupplierId INT,
    IN inProductId INT
)
BEGIN
    -- exit if the duplicate key occurs
    DECLARE EXIT HANDLER FOR 1062
    BEGIN
        SELECT CONCAT('Duplicate key (',inSupplierId,',',inProductId,') occurred') AS message;
    END;

    -- insert a new row into the SupplierProducts
    INSERT INTO SupplierProducts(supplierId,productId)
    VALUES(inSupplierId,inProductId);

    -- return the products supplied by the supplier id
    SELECT COUNT(*)
    FROM SupplierProducts
    WHERE supplierId = inSupplierId;
END |
DELIMITER ;
```

- The exit handler `DECLARE EXIT HANDLER FOR 1062` terminates the stored procedure whenever a duplicate key occurs. In addition, it returns an error message.
- Example stored procedure calls:
- The second time call to `InsertSupplierProduct(1,3)` invokes the `exit handler for 1062`
- `SELECT CONCAT('Duplicate key (',inSupplierId,',',inProductId,') occurred') AS message;` gets executed and procedure ends.

```
CALL InsertSupplierProduct(1,1);
CALL InsertSupplierProduct(1,2);
CALL InsertSupplierProduct(1,3);
CALL InsertSupplierProduct(1,3);
```

Exceptions - 10

```
DROP PROCEDURE IF EXISTS InsertSupplierProduct;
DELIMITER |
CREATE PROCEDURE InsertSupplierProduct(
    IN inSupplierId INT,
    IN inProductId INT
)
BEGIN
    -- exit if the duplicate key occurs
    DECLARE CONTINUE HANDLER FOR 1062
    BEGIN
        SELECT CONCAT('Duplicate key (',inSupplierId,',',inProductId,') occurred') AS message;
    END;

    -- insert a new row into the SupplierProducts
    INSERT INTO SupplierProducts(supplierId,productId)
    VALUES(inSupplierId,inProductId);

    -- return the products supplied by the supplier id
    SELECT COUNT(*)
    FROM SupplierProducts
    WHERE supplierId = inSupplierId;

END|
DELIMITER ;
```

Exceptions - 10 explanation

- The exception handler is changed from `exit` to `continue`
- Therefore the three stored procedure calls will execute without issue.
- Fourth stored procedure call issues message "Duplicate key..."
- However, the stored procedure continues to execute the last SQL statement `SELECT COUNT(*)`

- In case you have multiple handlers that handle the same error, MySQL will call the most specific handler to handle the error first based on the following rules:
  - An error always maps to a MySQL error code because in MySQL it is the most specific.
  - An `SQLSTATE` may map to many MySQL error codes, therefore, it is less specific.
  - An `SQLEXCPETION` or an `SQLWARNING` is the shorthand for a class of `SQLSTATES` values so it is the most generic.
- Based on the handler precedence rules, `MySQL` error code handler, `SQLSTATE` handler and `SQLEXCEPTION` takes the first, second and third precedence.

Exceptions - 12

```
DROP PROCEDURE IF EXISTS InsertSupplierProduct;

DELIMITER |
CREATE PROCEDURE InsertSupplierProduct(
    IN inSupplierId INT,
    IN inProductId INT
)
BEGIN
    -- exit if the duplicate key occurs
    DECLARE EXIT HANDLER FOR 1062 SELECT 'Duplicate keys error encountered' Message;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION SELECT 'SQLException encountered' Message;
    DECLARE EXIT HANDLER FOR SQLSTATE '23000' SELECT 'SQLSTATE 23000' ErrorCode;

    -- insert a new row into the SupplierProducts
    INSERT INTO SupplierProducts(supplierId,productId)
    VALUES(inSupplierId,inProductId);

    -- return the products supplied by the supplier id
    SELECT COUNT(*)
    FROM SupplierProducts
    WHERE supplierId = inSupplierId;

END|

DELIMITER ;
```

Exceptions - 13

- 1062: Duplicate keys error
- 23000: This issue occurs when more than one table are having the same column name and in a join query, the column name is being used without the alias of the table name.
- SQLEXCEPTION: general exception

Exceptions - 14 - named exceptions

```
DROP PROCEDURE IF EXISTS TestProc;
DELIMITER |
CREATE PROCEDURE TestProc()
BEGIN
    DECLARE TableNotFound CONDITION for 1146 ;

    DECLARE EXIT HANDLER FOR TableNotFound
        SELECT 'Please create table abc first' Message;
    SELECT * FROM abc;
END |
DELIMITER ;
```